

Drzewa przedziałowe

(drzewa licznikowe, zakresowe, interwałowe)

Umowna nazwa rodziny struktur danych, stosowanych do statystyk porządkowych i/lub operacji dotyczących zakresów (przedziałów) danych.

Drzewo licznikowe (drzewo multizbioru)

Założenie:

- ustalone $n = 2^k$,
- ustalone uniwersum $U = \{0, 1, 2, \dots, n-1\}$,
- multizbiór S elementów z uniwersum U .

Struktura podstawowa drzewa licznikowego n -elementowego:

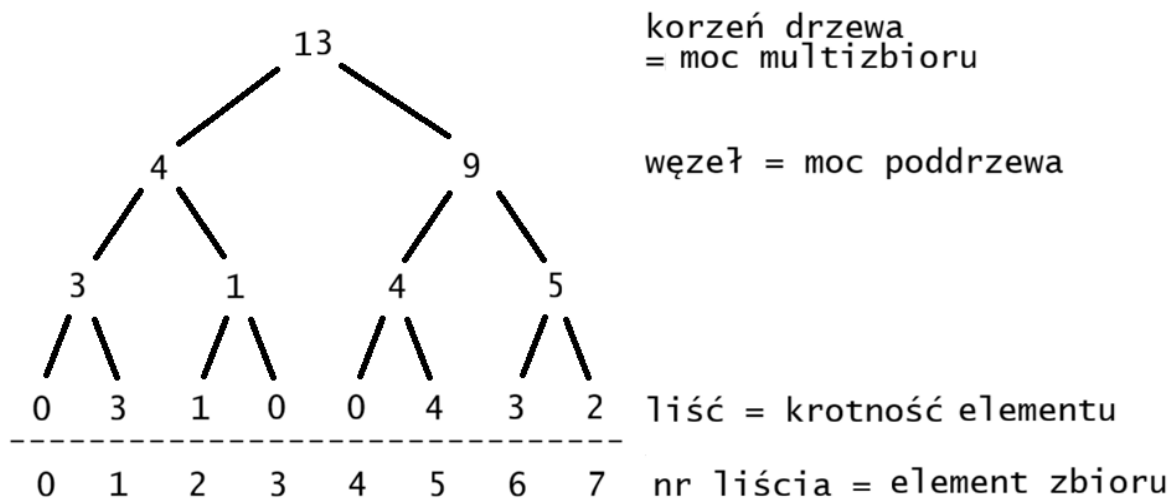
- drzewo binarne $(2n-1)$ -wierzchołkowe, w pełni zrównoważone,
- reprezentowane w tablicy $A[1..2n-1]$ jak kopiec,
- liście, czyli $A[n..2n-1]$, odpowiadają elementom zbioru S ,
- informacja o elemencie $x \in U$ jest w $A[n+x]$.

W każdym węźle drzewa informacja:

- dla $j \in [n, 2n-1]$, liść $A[j]$ = liczba wystąpień elementu $x = j - n$ w multizbiorze S ;
- $A[j]$, $j < n$ - liczba (wystąpień) elementów występujących w poddrzewie o korzeniu j (suma liści poddrzewa = liczność podzbioru złożonego z tych liści).

Przykład:

- $n = 8$, $U = \{0,1,2,3,4,5,6,7\}$ - uniwersum,
- $S = \{1,1,1,2,5,5,5,5,6,6,6,7,7\}$ - multizbiór, $|S|=15$,
- drzewo licznikowe multizbioru S w tablicy A :
 $A[1..15] = [13,4,9,3,1,4,5,0,3,1,0,0,4,3,2]$



Operacje na drzewie licznikowym:

1. Update(A, x, k)

zmień liczbę wystąpień elementu x o krotność k ;
 $k > 0$ - dodawanie, $k < 0$ - usuwanie.

na ścieżce od $A[n+x]$ do $A[1]$ zmień wartości o k ;

Złożoność $\Theta(\log n)$

Update() jest uogólnieniem Insert i Delete.

2. Min(A)

wyszukaj pierwszy od lewej dodatni liść:
od korzenia przesuwaj się w dół, do lewego jeśli
niezerowy, wpp. do prawego

Złożoność $\Theta(\log n)$

3. Next(A, x) następny w zbiorze (rosnąco)

```
// szukaj następny niezerowy liść:
i := n+x;      // adres liścia zawierającego element x
while i>1 do
    // idąc w górę przy pierwszej możliwości
    // przeskocz do brata na prawo
    if i%2=0 AND A[i+1]≠0 then
        // i parzyste <=> i jest lewym następnikiem
        i++; break;
    else
        i := i/2;
if i=1 then return NULL;
while i<n do
    // szukaj minimum w poddrzewie o korzeniu i,
    // wiemy że korzeń jest niepusty
    // wybieramy niepusty następnik,
    // z preferencją w lewo
    i := 2*i;
    if A[i]=0 then i++;
return i-n;     // numer (nazwa) znalezionej elementu
```

Złożoność $\Theta(\log n)$

4. Kth(A, k) k-ty element w zbiorze (rosnąco)

```
if k > A[1] then
    return NULL;
i := 1;
while i < n do
    i := 2*i;
    if A[i] < k then
        k := k - A[i];
        i++;
```

return i-n; Złożoność $\Theta(\log n)$

5. RangeQuery(A, x, y)

podaj liczbę elementów z: $x \leq z \leq y$.
(klasyczne zapytanie przedziałowe)

Skonstruujemy efektywny algorytm dla tego zapytania z użyciem **przedziałów bazowych**.

Definicja (przedział bazowy)

Przedział $[p, q]$, gdzie $n \leq p \leq q \leq 2n-1$, jest przedziałem bazowym w drzewie licznikowym n -elementowym o strukturze podstawowej T jeśli

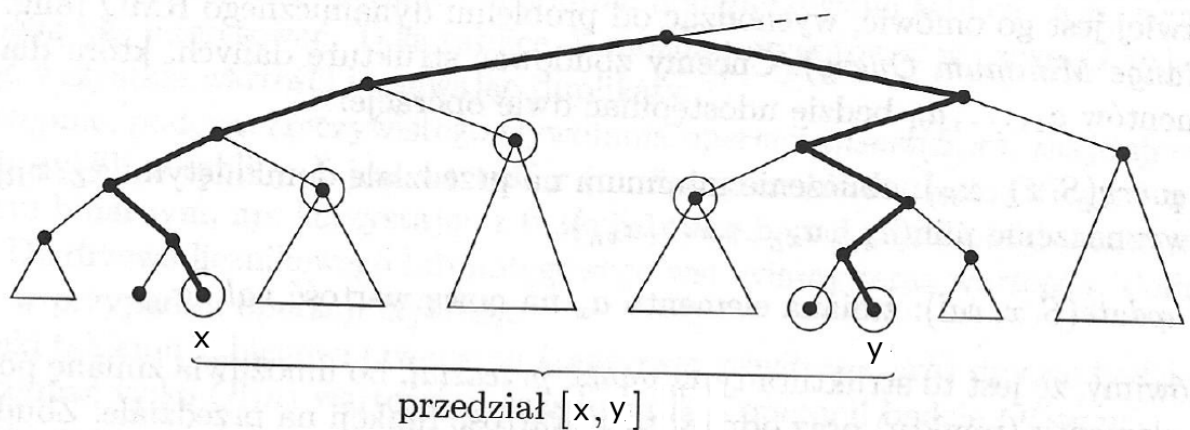
- węzły $A[p], A[p+1], \dots, A[q]$ stanowią zbiór wszystkich liści pewnego poddrzewa drzewa T .

Algorytm dla zapytania RangeQuery() ustala wszystkie maksymalne (w sensie inkluzji) przedziały bazowe, których suma mnogościowa stanowi przedział $[x, y]$;

- są one parami rozłączne, jest ich $O(\log n)$.

Do obliczenia odpowiedzi na zapytanie (ilościowe) wystarcza

- znaleźć korzenie tych przedziałów bazowych oraz
- obliczyć ich sumę (pytanie o sumę po zakresie).



1. znajdź m = najniższy wspólny przodek x i y ;
 - x i y są na tej samej głębokości, więc
 - wystarczy równolegle iść dwiema ścieżkami do góry
2. na ścieżce od m do x oznakuj prawe następniki węzłów w których schodzimy w lewo;
3. na ścieżce od m do y oznakuj lewe następniki węzłów w których schodzimy w prawo;
4. oznakuj x i y ;
5. //węzły oznakowane to korzenie przedziałów bazowych
 oblicz sumę węzłów oznakowanych.

Pseudokod w postaci jednej pętli idącej dwiema ścieżkami w górę, liczącej sumę na bieżąco:

```

RangeQuery(A, x, y)
// metoda iteracyjna, od obu liści w górę drzewa
x:=x+n; y:=y+n;          // początek w liściach
val:=A[x]+A[y];
while  $\lfloor x/2 \rfloor \neq \lfloor y/2 \rfloor$  do    // x i y nie są braćmi
  if  $x \% 2 = 0$  then          // x jest lewym dzieckiem
    val:=val+A[x+1];          // prawy brat
  if  $y \% 2 = 1$  then          // y jest prawym dzieckiem
    val:=val+A[y-1];          // lewy brat
  x:= $\lfloor x/2 \rfloor$ ; y:= $\lfloor y/2 \rfloor$ ;
return val;
end;
```

Złożoność $\Theta(\log n)$.

Popularna jest również wersja rekurencyjna:

```
main
  x,y:=lewy i prawy koniec przedziału domkniętego;
  root:=1;
  answer:=RangeQuery(A, root, 0, n, x, y+1);
end;

RangeQuery(A, node, L, R, x, y)
// node - korzeń poddrzewa przedziału bazowego [L,R)
// [x,y) - zakres zapytania,
// oba przedziały prawostronnie otwarte !!!
  if y≤L or R≤x then
    // zakres zapytania i przedział bazowy rozłączne
    return 0;    // element neutralny dodawania
  if x≤L and R≤y then
    // przedział bazowy zawarty w zakresie zapytania
    return A[node];
  else
    M:=⌊(L+R)/2⌋;
    v:=RangeQuery(A, 2*node, L, M, x, y);
    w:=RangeQuery(A, 2*node+1, M, R, x, y);
    return v+w;
end.
```

Uwagi:

- dowód poprawności wersji rekurencyjnej jest łatwy;
- uzasadnienie złożoności $\Theta(\log n)$ sprowadza się do wykazania, że wersja rekurencyjna wykrywa te same przedziały bazowe co wersja iteracyjna;
- posługiwanie się w wersji rekurencyjnej przedziałami półotwartymi (zarówno dla zakresu zapytania jak i przedziału bazowego) ułatwia modyfikacje granic przedziałów.

Popularne warianty drzewa licznikowego:

Drzewo maksimumów:

- każdy element (liść) ma przypisaną wartość,
- każdy węzeł wewnętrzny przechowuje maksimum wartości liści w swoim poddrzewie.

Operacje:

- zmień wartość elementu o podanym numerze (+/-)
aktualizacja ścieżki od elementu do korzenia,
wyliczając dla kolejnych węzłów maksimum z
lewego i prawego następnika;
- podaj maksimum wartości elementów z zadanego przedziału (numerów) elementów [x,y];
identycznie jak RangeQuery() dla drzewa multizbioru.

Drzewo sum (drzewo zliczania)

Analogicznie, zastępując maksimum przez sumę.

Drzewa przedziałowe - klasyfikacja podstawowa

Typ 1: punkt - przedział

- operacje (modyfikacja) na pojedynczych elementach ("punktach"),
- pytania (przynajmniej niektóre) dotyczące przedziałów.

Znany, klasyczny problem:

RMQ - range minimum query, wersja dynamiczna.

Dane: ciąg elementów $a_1, \dots, a_n$.

Operacje:

- $\text{query}(i, j)$: wylicz $\min\{a_i, \dots, a_j\}$.
- $\text{update}(i, \text{val})$: $a_i := \text{val}$.

Schemat realizacji jak dla multizbioru (lub sumy, lub maksimum).

Typ 2: przedział - punkt

(operacje na przedziale, pytania dotyczące punktu)

Np. w każdym węźle wewnętrznym jest modyfikator wartości każdego elementu z poddrzewa danego węzła.

Operacje:

- odczytaj wartość elementu o podanym numerze,
- zmień wartość wszystkich elementów z danego przedziału o zadaną wielkość.

Typ 3: przedział - przedział

(operacje na przedziale, pytania dotyczące przedziału)

Poza zakresem tego wykładu.